



## LeetCode 202: Happy Number

### 1. Problem Title & Link

**Title:** LeetCode 202: Happy Number

**Link:** <https://leetcode.com/problems/happy-number/>

### 2. Problem Statement (Short Summary)

A number is called a **Happy Number** if:

1. Replace the number with the sum of the squares of its digits.
2. Repeat the process.
3. If the number eventually becomes 1, it is happy.
4. If the process enters a cycle and never reaches 1, it is not happy.

Return:

true -> Happy Number

false -> Not Happy Number

### 3. Examples (Input → Output)

#### Example 1

Input:

$n = 19$

Process:

19

↓

82

↓

68

↓

100

↓

1

Output:

true

#### Example 2

Input:

$n = 2$

Process:

2



↓

4

↓

16

↓

37

↓

58

↓

89

↓

145

↓

42

↓

20

↓

4

Cycle detected.

Output:

false

**Example 3**

Input:

n = 1

Output:

true

**4. Constraints** $1 \leq n \leq 2^{31} - 1$ 

Important Observation:

Process can repeat forever.

Need cycle detection.

**5. Core Concept (Pattern / Topic)****HashSet / Cycle Detection**

Secondary Topics:



- Mathematics
- Digit Extraction
- Fast & Slow Pointer (Alternative)

## 6. Thought Process (Step-by-Step Explanation)

### Step 1: Generate Next Number

For:

19

Calculate:

$$1^2 + 9^2$$

$$= 1 + 81$$

$$= 82$$

For:

82

Calculate:

$$8^2 + 2^2$$

$$= 64 + 4$$

$$= 68$$

Continue until:

1

or a cycle appears.

### Step 2: Detect Infinite Loop

Example:

2

↓

4

↓

16

↓

37

↓



58

↓

89

↓

145

↓

42

↓

20

↓

4

Notice:

4 appears again

Once a number repeats:

The sequence will repeat forever.

So:

Return false

### Key Insight

Two possibilities:

Reach 1

OR

Enter a cycle

To detect cycles:

Store previously seen numbers in a HashSet.

### 7. Visual / Intuition Diagram (ASCII Diagram)

```
Happy Number
```

```
19
```

```
↓
```

```
82
```

```
↓
```

```
68
```

```
↓
```

```
100
```

```
↓
```

```
1
```

```
Result:
```

```
Happy Number ✓
```



Non-Happy Number

2

↓

4

↓

16

↓

37

↓

58

↓

89

↓

145

↓

42

↓

20

↓

4

Back to:

4

Cycle detected.

Result:

Not Happy Number ✗

## 8. Pseudocode (Language Independent)

```
create seen set

while n != 1

    if n already in set

        return false

    add n to set

    n = sumOfSquareDigits(n)

return true
```

## Helper Function

```
sum = 0

while n > 0

    digit = n % 10
```



```
sum += digit * digit

n = n / 10

return sum
```

## 9. Code Implementation

### Python

```
class Solution:

    def isHappy(self, n: int) -> bool:

        seen = set()

        while n != 1:

            if n in seen:
                return False

            seen.add(n)

            total = 0

            while n > 0:

                digit = n % 10
                total += digit * digit
                n //= 10

            n = total

        return True
```

### Java

```
import java.util.*;

class Solution {

    public boolean isHappy(int n) {

        HashSet<Integer> seen =
            new HashSet<>();

        while(n != 1) {

            if(seen.contains(n))
```



```
        return false;

    seen.add(n);

    int total = 0;

    while(n > 0) {

        int digit = n % 10;

        total += digit * digit;

        n /= 10;
    }

    n = total;
}

return true;
}
```

## 10. Time & Space Complexity

### Time Complexity

$O(\log n)$

Digit extraction takes:

$O(\text{number of digits})$

and the sequence quickly becomes small.

### Space Complexity

$O(\log n)$

For storing visited numbers.

## 11. Common Mistakes / Edge Cases

### Mistake 1

Not detecting cycles.

Leads to:

Infinite Loop

### Mistake 2

Using:

$\text{digit} * 2$



instead of:

digit \* digit

### Mistake 3

Not resetting total.

Wrong:

total += ...

across iterations.

### Edge Cases

**n = 1**

Output:

true

**n = 7**

Process:

7

↓

49

↓

97

↓

130

↓

10

↓

1

Happy Number.

## 12. Detailed Dry Run

Input:

n = 19

| Current<br>n | Calculatio<br>n | Next<br>n |
|--------------|-----------------|-----------|
| 19           | $1^2 + 9^2$     | 82        |
| 82           | $8^2 + 2^2$     | 68        |
| 68           | $6^2 + 8^2$     | 100       |





|     |                   |   |
|-----|-------------------|---|
| 100 | $1^2 + 0^2 + 0^2$ | 1 |
|-----|-------------------|---|

Reached:

1

Return:

true

### Dry Run for n = 2

| Current<br>n | Next<br>n |
|--------------|-----------|
| 2            | 4         |
| 4            | 16        |
| 16           | 37        |
| 37           | 58        |
| 58           | 89        |
| 89           | 145       |
| 145          | 42        |
| 42           | 20        |
| 20           | 4         |

Already seen:

4

Cycle detected.

Return:

false

## 13. Common Use Cases (Real-Life / Interview)

### Cycle Detection Problems

Used in:

Repeated State Detection

Loop Detection

### Mathematical Simulations

Repeated transformations.

### HashSet Applications

Track visited states.

### Interview Purpose



Tests:

HashSet

Digit Extraction

Cycle Detection

#### 14. Common Traps (Important!)

##### Trap 1

Ignoring repeated values.

Infinite loop occurs.

##### Trap 2

Using List instead of Set.

Lookup becomes:

$O(n)$

instead of:

$O(1)$

##### Trap 3

Confusing:

Digit Sum

with

Sum of Squares of Digits

#### 15. Builds To (Related LeetCode Problems)

LC 202 - Happy Number



LC 141 - Linked List Cycle



LC 142 - Linked List Cycle II



LC 287 - Find Duplicate Number



LC 457 - Circular Array Loop

All involve cycle detection concepts.

#### 16. Alternate Approaches + Comparison

| Approach | Time | Space | Interview Rating |
|----------|------|-------|------------------|
|----------|------|-------|------------------|



|                       |             |             |                    |
|-----------------------|-------------|-------------|--------------------|
| HashSet               | $O(\log n)$ | $O(\log n)$ | Best for Beginners |
| Floyd Cycle Detection | $O(\log n)$ | $O(1)$      | Advanced           |

### Approach 1: HashSet (Chosen)

Store all visited numbers.

Easy to understand.

### Approach 2: Floyd Cycle Detection

Exactly like Linked List Cycle.

Use:

slow

fast

pointers.

Space:

$O(1)$

### 17. Why This Solution Works (Short Intuition)

Every sequence must either:

Reach 1

or

Repeat a previous number

By storing seen values, we can immediately detect cycles and determine whether the number is happy.

### 18. Variations / Follow-Up Questions

#### Follow-Up 1

Return the number of steps needed to reach 1.

#### Follow-Up 2

Print the complete happy sequence.

Example:

$19 \rightarrow 82 \rightarrow 68 \rightarrow 100 \rightarrow 1$

#### Follow-Up 3

Solve using Floyd's Cycle Detection.

Space:

$O(1)$



### Follow-Up 4

Check happiness in:

Binary

Hexadecimal

Other Bases

### Interview Takeaway

The key pattern is:

Visited State Tracking

Whenever you hear:

Repeated transformations

Infinite loops

Cycle detection

think:

HashSet

or

Fast & Slow Pointer

This concept appears frequently in linked lists, graphs, simulations, and state-machine problems.